CSS4005 — Cloud Infrastructure Construction

Lecture 9: Load Balancing and Auto Scaling

Nauruzbayeva Farikha · Lesson 9



Recap — Lesson 8 (AWS Architecture)

- What are the three tiers of a classic web architecture? (web/entry point, application, data)
- What is a single point of failure? (one component whose failure takes the whole service down — e.g. a single EC2 instance)
- Which Well-Architected pillar asks "does it keep working when parts fail"? (Reliability)
- What box did we leave as a placeholder last week? (the load balancer in front of the app tier)

Today’s agenda

- ☐ Why load balance: availability and scale
- ☐ Elastic Load Balancing: ALB, NLB, GWLB — and choosing one
- ☐ Inside an ALB: listeners, rules, target groups, health checks
- ☐ Auto Scaling: launch templates, groups, scaling policies
- ☐ ALB + Auto Scaling together: surviving failures and spikes
- ☐ Cost and common mistakes → today’s practice

The scenario

Last week you designed NarxozReg, the course-registration app. On a normal day, two servers are plenty. Then registration opens at 09:00 and 5,000 students press the button in the same minute.

One server is overloaded, another one crashes, and nobody is awake to launch a third.

Today: one address in front, a fleet behind it that grows, shrinks and repairs itself.

Block 1

Why load balance

One server is two problems

- **Availability:** if the only server dies, the service is down. A single EC2 instance is the textbook single point of failure.
- **Capacity:** one server has a fixed amount of CPU and memory. A bigger instance type helps only up to a point, and needs downtime to change.
- **Fixed address:** clients connect to *one* IP. Replace the server and the address changes (week 7: the public IP changes on stop/start).

The fix for all three: put a **load balancer** in front of several identical servers. Clients talk to the load balancer; it picks a healthy server for every request.

Scale up vs scale out

	Scale **up** (vertical)	Scale **out** (horizontal)
How	Bigger instance type	More instances of the same type
Limit	Largest instance available	Practically none
Downtime to change	Usually yes (stop → change type → start)	No — add or remove servers live
Survives a server failure	No	Yes, the others keep serving
Needs	Nothing special	A load balancer + a **stateless** app

Cloud architectures prefer scaling **out**. Today's tools — Elastic Load Balancing and EC2 Auto Scaling — exist to make that easy.

Block 2

Elastic Load Balancing

Three kinds of load balancer

	Application LB (ALB)	Network LB (NLB)	Gateway LB (GWLB)
Layer	7 — HTTP / HTTPS	4 — TCP / UDP / TLS	3 — IP packets
Routes by	Path, host, headers, query	IP and port	Sends traffic through appliances
Typical use	Web apps, APIs, microservices	Very high throughput, non-HTTP, static IPs	Third-party firewalls and inspection

- All three are **managed**: AWS runs and scales them across AZs for you.
- Rule of thumb for this course: **web traffic → ALB**. NLB when you need raw TCP/UDP or fixed IPs. GWLB you'll meet only with security appliances.

Anatomy of an ALB

- **Load balancer:** one DNS name (e.g. `my-alb-123456.us-east-1.elb.amazonaws.com`), placed in subnets in **at least two AZs**.
- **Listener:** a port and protocol it accepts, e.g. HTTP:80, HTTPS:443.
- **Rules:** on each listener, "if the request looks like X, send it to Y". There's always a default rule.
- **Target group:** the set of servers (instances, IPs or Lambda functions) that receive the traffic, with a **health check**.

Request flow: client → DNS name → listener → rule → target group → one **healthy** target.

Listener rules: one ALB, many apps

```
Listener HTTP:80
  rule 1  path   /api/*            → target group  api-servers
  rule 2  host   admin.narxoz.kz   → target group  admin-servers
  default                          → target group  web-servers
```

- **Path-based routing:** different URL paths go to different services.
- **Host-based routing:** different domain names share one ALB.
- Other conditions: HTTP headers, method, query string, source IP.
- Rules are checked by priority; the first match wins.

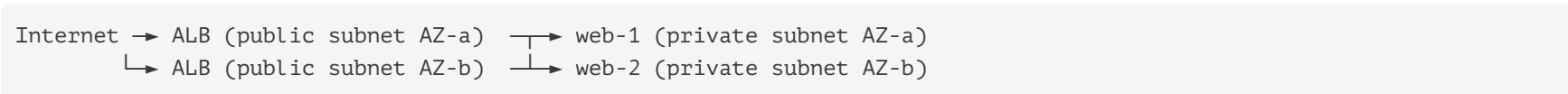
Health checks

- The ALB regularly sends a request to every target, e.g. `GET /health`, and expects a success code (200 by default).
- **Unhealthy threshold:** after this many failed checks in a row, the target is taken out of rotation. **Healthy threshold:** after this many successes, it's put back.
- ALB defaults: a check every 30 s, timeout 5 s, 5 successes to become healthy, 2 failures to become unhealthy — all adjustable per target group.
- Traffic only goes to **healthy** targets. If none are healthy, the ALB returns **503**.

A health-check path that returns 404 or 302 marks every server unhealthy, even though the site "works" in your browser.

Where the ALB lives

- The ALB goes in **public subnets** in two or more AZs — it's the only thing the internet talks to.
- The targets go in **private subnets** (week 4, week 8): no public IPs, reachable only through the ALB.
- **Cross-zone load balancing** spreads requests evenly across *all* healthy targets in all AZs. It's on by default for ALB.



Security groups, chained again

Security group	Inbound rule	Source
alb-sg	TCP 80 (and 443)	`0.0.0.0/0` — the internet
web-sg	TCP 80	**alb-sg** — only the load balancer
db-sg	TCP 5432	**web-sg** — only the app servers

- Same pattern as week 7 (SG-DB ← SG-APP), one tier higher.
- Nobody on the internet can reach a web server directly, even if it had a public IP: web-sg only trusts the ALB.

Sticky sessions — and why to avoid needing them

- **Sticky sessions** (session affinity): the ALB sets a cookie and keeps sending the same user to the same target.
- Useful when a server keeps a user's session **in its own memory**.
- The catch: that server dies → the user's session is gone; load becomes uneven; scaling in kicks users out.
- Better: make the app **stateless** — keep sessions in a shared store (a database, a cache) so *any* server can answer *any* request.

> Stateless servers are what make load balancing and auto scaling simple.

HTTPS at the load balancer

- Add an **HTTPS:443 listener** with a TLS certificate. AWS Certificate Manager (**ACM**) issues public certificates for use with ELB at no extra charge, and renews them automatically.
- The ALB **terminates TLS**: browser ↔ ALB is encrypted; the ALB then forwards to the targets (over HTTP inside the VPC, or HTTPS again if required).
- Common pattern: the HTTP:80 listener only **redirects** to HTTPS:443.

Quick check

Every target in your target group is **unhealthy**, but when you open a server's IP in a browser you see the site. Name two likely causes.

The health-check path doesn't return 200 (wrong path, redirect, 404), or the web servers' security group doesn't allow the port from the ALB's security group.

Block 3

EC2 Auto Scaling

The idea

- An **Auto Scaling group (ASG)** keeps a fleet of identical EC2 instances at the size you ask for.
- It **replaces** instances that fail their health checks — self-healing.
- It **changes the size** automatically from a metric (CPU, requests) or a schedule — elasticity.
- Auto Scaling itself has no extra charge: you pay for the instances it runs.

Launch template: the recipe

- Everything needed to launch one instance, saved and **versioned**:
  - AMI (e.g. the latest Amazon Linux 2023)
  - instance type (e.g. `t3.micro`)
  - security groups (web-sg)
  - user data (install nginx, deploy the app — week 7)
  - instance profile (IAM role, week 6), key pair, IMDSv2 setting
- Every instance the ASG launches comes from this recipe, so they're identical and replaceable.

If an instance needs a human to log in and fix it before it works, it can't be auto scaled. User data and AMIs make servers disposable.

The Auto Scaling group

Setting	Meaning	Example
Min	Never fewer than this	2
Desired	How many to run right now	2
Max	Never more than this (a cost guard)	6
Subnets	Where to launch — pick **two or more AZs**	private-a, private-b
Target groups	Register new instances with the ALB automatically	web-tg
Health check type	EC2 only, or also the **ELB** health check	ELB

With **ELB** health checks, an instance that's running but serving errors is also replaced — not just one that crashed.

Scaling policies

- **Target tracking** (start here): "keep average CPU at 50%". The ASG creates the CloudWatch alarms and adds or removes instances to stay near the target.
- **Step scaling:** "CPU > 70% → add 2; CPU > 90% → add 4". More control, more to configure.
- **Scheduled scaling:** "every weekday at 08:30, desired = 6; at 20:00, desired = 2". Perfect for known peaks like registration week.
- Policies only move **desired** — always within min and max.

Cooldowns, warm-up and instance refresh

- A new instance needs time to boot and run user data before its metrics mean anything. **Warm-up / cooldown** settings stop the ASG from reacting again too soon and overshooting.
- **Health check grace period:** how long after launch the ASG ignores failed health checks while the app starts.
- **Instance refresh:** changed the launch template (new AMI, new user data)? Instance refresh replaces running instances a few at a time, keeping a minimum percentage healthy — a rolling deployment.

Quick check

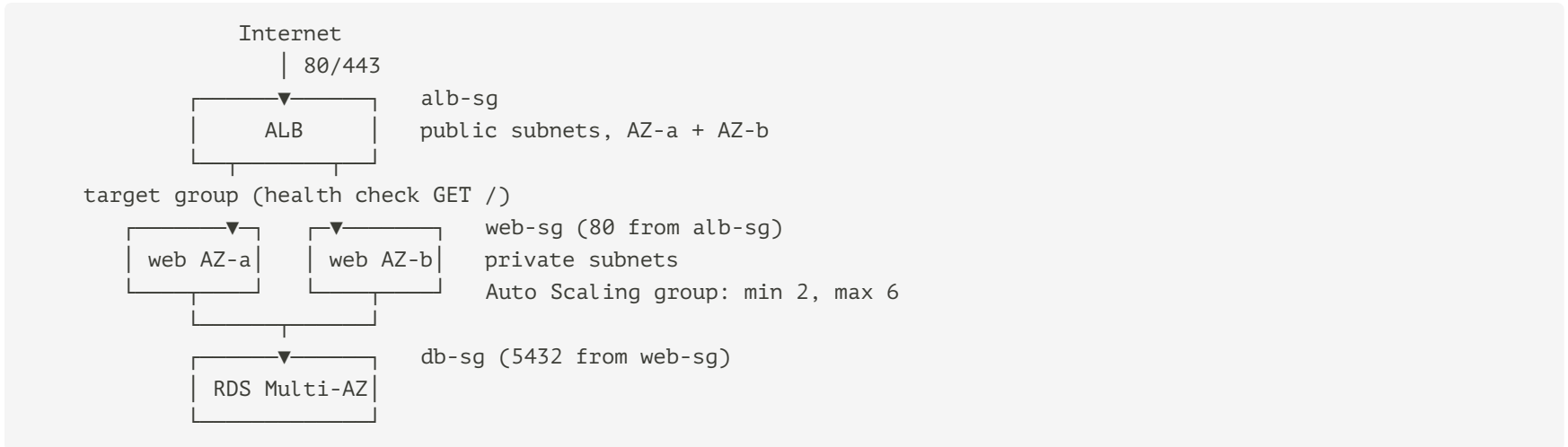
An ASG has min 2, desired 2, max 4. You terminate one instance by hand. What happens next, and why?

The ASG sees only 1 running instance, below desired 2, and launches a replacement from the launch template. It registers with the target group and gets traffic once it passes the health check.

Block 4

Putting it together

ALB + Auto Scaling group



Two jobs, two services

Event	The ALB…	The Auto Scaling group…
A server stops responding	Stops sending it traffic after the health check fails	Terminates it and launches a replacement
Traffic doubles	Spreads requests over all healthy targets	Raises desired (policy) and registers new instances
Traffic drops at night	Keeps routing to whoever is left	Lowers desired, deregisters and terminates extras
You deploy a new version	Routes around instances being replaced	Instance refresh replaces them a few at a time

The ALB decides **where requests go**. The ASG decides **how many servers exist**. You need both.

Walk-through: an AZ goes down

1. Instances in AZ-a stop answering health checks.
2. The ALB marks them unhealthy and sends **all** traffic to AZ-b (the ALB itself also runs in both AZs).
3. The ASG notices fewer healthy instances than desired and launches replacements — in the AZs that still work.
4. RDS Multi-AZ fails over to its standby (week 7) — same endpoint.
5. Users may notice a short slowdown, not an outage.

That's the "survive one AZ failure" requirement from last week's practice, fully filled in.

What’s still a single point of failure?

- **The app’s state**, if sessions live in one server’s memory → stateless app, shared session store.
- **A single NAT gateway** for both private subnets → one NAT per AZ.
- **The launch template / AMI**: a broken new version can replace your whole fleet → instance refresh with a minimum healthy percentage, and keep the old version to roll back.
- **Your max size**: set it too low and the spike still wins; too high and the bill wins.

Cost hygiene

- An ALB is billed **for every hour it exists**, plus **LCUs** (Load Balancer Capacity Units) for the traffic it handles — even with zero requests, the hourly charge runs.
- Auto Scaling is free; the **instances** it launches are not. **Max** is your spending limit.
- In a lab: delete the **ASG first** (or it keeps replacing what you delete), then the ALB, then the target group and launch template.
- Check current prices on the Elastic Load Balancing pricing page; they differ by region.

Common mistakes

Symptom	Likely cause
All targets unhealthy	Health-check path returns 404/302, or web-sg doesn't allow the ALB
ALB returns 503	No healthy targets yet (still booting, user data not finished)
`curl` to the ALB hangs	ALB still provisioning, or alb-sg missing the inbound rule
All traffic in one AZ	ASG subnets all in the same AZ
Instances keep coming back after you delete them	You deleted instances, not the ASG
New instances fail the check and get replaced in a loop	Grace period too short for user data to finish

Quick check

Registration opens every semester on a known date at 09:00. Which scaling policy fits best, and why not only target tracking?

Scheduled scaling: raise desired before 09:00. Target tracking only reacts after CPU is already high, and new instances need minutes to boot — use both: a schedule for the known peak, target tracking for surprises.

Block 5

Try it yourself — today's practice

Today’s practice — a self-healing fleet

Track A — AWS Academy Learner Lab (`us-east-1`)

1. alb-sg and web-sg (80 only from alb-sg)
2. Launch template with nginx user data
3. Target group + ALB in two AZs
4. ASG: min 2, desired 2, max 4
5. Kill an instance — watch it come back
6. Target-tracking policy: CPU 50%

Track B — no AWS (Docker)

1. HAProxy as the load balancer, with health checks
2. Three nginx containers behind it
3. Stop one — traffic continues
4. Crash one — `restart: always` brings it back
5. `--scale web=5` and back to 2: HAProxy finds them through Docker DNS

Checked orally: show requests answered by different servers, the health status, a server dying while the site keeps answering 200 — and your cleanup.

By the end of this lesson, you should be able to

- ☐ Explain why a load balancer improves both availability and capacity
- ☐ Choose between ALB, NLB and GWLB for a given workload
- ☐ Describe listeners, rules, target groups and health checks
- ☐ Configure an Auto Scaling group from a launch template, across two AZs
- ☐ Pick a scaling policy (target tracking, step, scheduled) for a scenario
- ☐ Explain how ALB + Auto Scaling survive an instance or AZ failure

Before next lecture

- ☐ Finish today's practice and make sure **everything is deleted** in Learner Lab (ASG, ALB, target group, launch template)
- ☐ Add the ALB and the Auto Scaling group to last week's NarxozReg diagram
- ☐ Install Docker Desktop (or Docker / Podman on Linux) — we use it next week

Next week: packaging the app itself, so every server — and every laptop — runs exactly the same thing.

Next lecture

Docker and Containers — build an image once, run it anywhere.